Inyección de dependencias en NestJS

Qué es, cómo funciona por dentro y cómo la usa el repo del profe (`icesi-2026b-nestjs`, car-dealershop)

1. La idea en una frase

Una clase no crea lo que necesita: lo pide, y alguien externo se lo entrega ya construido. Ese "alguien externo" en Nest se llama **contenedor de inyección de dependencias** (contenedor IoC, *Inversion of Control*). Tú declaras qué necesitas en el constructor; Nest decide cómo crearlo, cuándo, y con qué.

Una analogía: en un restaurante el cocinero no va al mercado a comprar cada ingrediente, ni fabrica la sartén. Escribe en su hoja "*necesito tomates y una sartén*" y el restaurante se los pone en la mesa. Si mañana cambian de proveedor de tomates, el cocinero ni se entera.

2. El problema que resuelve

Imagina `CarsService` escrito **sin** inyección. Necesita hablar con la base de datos y con las marcas, así que las crea él mismo:

```
SIN inyección (así NO se hace en Nest)

export class CarsService {
    private carRepository = new Repository(Car, ...);  // ¿cómo conecta a Postgres?
    private brandService  = new BrandsService();       // y este necesita SU repositorio
}
```

- **Acoplamiento:** `CarsService` tiene que saber *cómo* se construye todo lo que usa (y lo que ese algo usa, y así sucesivamente).
- **Duplicados:** cada `new` crea otra instancia; podrías tener 5 `BrandsService` con 5 conexiones distintas.
- **Imposible de probar:** en un test no puedes cambiar la base de datos real por una falsa.
- **Cambios en cascada:** si `BrandsService` pasa a necesitar otro parámetro, hay que arreglar cada sitio donde se hace `new`.

Con inyección, `CarsService` solo **declara** lo que necesita y Nest se ocupa del resto:

```
cars.service.ts (código real del repo)

@Injectable()
export class CarsService {
    constructor(
        @InjectRepository(Car) private readonly carRepository: Repository<Car>,
        private readonly brandService: BrandsService,
    ) { }
}
```

3. Las tres piezas

Pieza	Qué es	En el repo
Provider (lo que se inyecta)	Una clase marcada con `@Injectable()` y registrada en el `providers` de un módulo. Nest guarda **una instancia** de ella.	`CarsService`, `BrandsService`, `UserService`, `AuthService`, `JwtStrategy`
Consumidor (quien lo pide)	Cualquier clase que gestione Nest y declare la dependencia en su **constructor**: controllers, services, guards, strategies.	`CarsController` pide `CarsService`; `UserRoleGuard` pide `Reflector`
Contenedor (quien entrega)	El mecanismo interno de Nest. Lee los módulos, construye el grafo de dependencias, crea las instancias en el orden correcto y las reparte.	Se dispara con `NestFactory.create(AppModule)` en `main.ts`

El **token** es la "etiqueta" con la que el contenedor identifica cada provider. Para una clase normal, el token es la clase misma (`BrandsService`). Para los repositorios de TypeORM es un texto derivado de la entidad (por eso los errores dicen `CarRepository`). Cuando escribes `private readonly brandService: BrandsService`, le estás diciendo a Nest: "*dame lo que esté registrado bajo el token* `BrandsService`".

4. Cómo lo hace Nest por dentro

Hay un truco de TypeScript detrás, y por eso el `tsconfig.json` del repo tiene dos líneas imprescindibles:

```
tsconfig.json

"experimentalDecorators": true,  // permite escribir @Injectable(), @Controller()...
"emitDecoratorMetadata": true,   // TypeScript conserva los TIPOS del constructor
```

Normalmente TypeScript borra los tipos al compilar. Pero si la clase tiene un decorador y `emitDecoratorMetadata` está activo, TypeScript deja escrito, para esa clase, algo equivalente a:

```
Reflect.defineMetadata("design:paramtypes", [Repository, BrandsService], CarsService);
//   los tipos de los parámetros del constructor, en orden, quedan guardados
```

Nest lee esa lista con `reflect-metadata` (dependencia del repo) y ya sabe qué pedirle al contenedor. Ese es el motivo por el que `@Injectable()` es necesario: sin ningún decorador sobre la clase, TypeScript no emite esa metadata y Nest no puede adivinar los tipos.

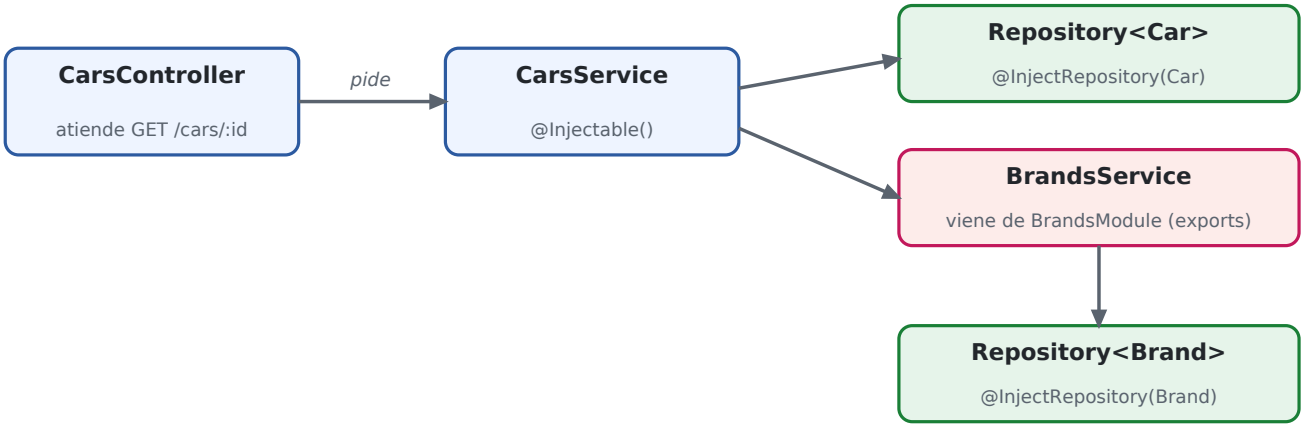
El arranque, paso a paso

#	Qué hace Nest	Ejemplo en el repo
1	**Escanea los módulos** empezando por el raíz y siguiendo cada `imports`.	`AppModule` → Cars, Brands, User, Auth, Config, TypeOrm
2	**Registra los providers** de cada módulo con su token.	`providers: [CarsService]`, más los repositorios que crea `forFeature`
3	**Resuelve dependencias** recursivamente: para crear A necesita B, para B necesita C...	CarsService → BrandsService → Repository<Brand>
4	**Instancia de hojas hacia raíz** (primero lo que no depende de nada) y guarda cada instancia (**singleton**).	Se crea `Repository<Brand>`, luego `BrandsService`, luego `CarsService`
5	**Instancia los controllers** inyectándoles los providers ya creados.	`CarsController` recibe el `CarsService` ya listo
6	**Registra las rutas** y abre el puerto.	`app.listen(3000)`

> **Singleton por defecto:** solo existe **una** instancia de `BrandsService` en toda la app, aunque la usen `BrandsController` y `CarsService`. Ambos reciben el *mismo objeto*. Todo esto ocurre **al arrancar**, no en cada petición: por eso un error de inyección aparece de inmediato y la app ni siquiera abre el puerto.

5. Seguimos un caso real: `GET /cars/:id`

Este es el grafo de dependencias que Nest construye para `CarsController`:



La flecha significa "necesita". Nest construye de derecha a izquierda: primero lo que no depende de nadie.

Cada caja sale de una línea concreta del repo:

Clase	Lo que pide en el constructor	De dónde lo saca Nest
`CarsController`	`private readonly carService: CarsService`	`providers: [CarsService]` de `CarsModule`
`CarsService`	`@InjectRepository(Car) ... Repository<Car>`	`TypeOrmModule.forFeature([Car])` en `CarsModule`
`CarsService`	`private readonly brandService: BrandsService`	`imports: [BrandsModule]` en Cars **y** `exports: [BrandsService]` en Brands
`BrandsService`	`@InjectRepository(Brand) ... Repository<Brand>`	`TypeOrmModule.forFeature([Brand])` en `BrandsModule`

Y luego, cuando llega la petición, el trabajo ya está montado. El controller simplemente **usa** lo que recibió:

```
cars.controller.ts / cars.service.ts

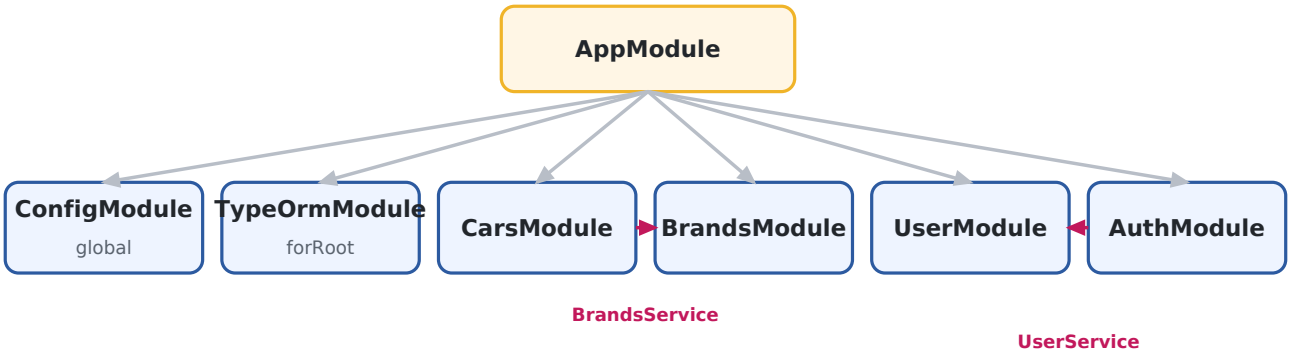
@Get(':id')
getById(@Param('id', ParseUUIDPipe) id: string){
    return this.carService.getById(id);   // carService ya existe: Nest lo inyectó
}

// dentro de CarsService.create:
const brand = await this.brandService.findOne(car.brand);  // el mismo singleton
```

6. Los módulos: quién puede ver qué

Los providers son **privados de su módulo**. Aunque `BrandsService` exista, `CarsModule` no lo ve a menos que se cumplan dos condiciones a la vez:

- `BrandsModule` lo **publica** con `exports: [BrandsService]`.
- `CarsModule` **importa** a `BrandsModule` en su `imports`.



Flechas rosas = un módulo IMPORTA a otro para usar el service que ese otro EXPORTA.

```
las dos mitades del "apretón de manos"

// brands.module.ts                          // cars.module.ts
@Module({                                    @Module({
  controllers: [BrandsController],             controllers: [CarsController],
  providers:   [BrandsService],                providers:   [CarsService],
  imports:     [TypeOrmModule.forFeature(      imports: [
                  [Brand])],                       TypeOrmModule.forFeature([Car]),
  exports:     [BrandsService]   // ← publica     BrandsModule,     // ← importa
})                                             ]
export class BrandsModule {}                 })
                                             export class CarsModule {}
```

El mismo patrón se repite para el login: `UserModule` tiene `exports: [UserService]` y `AuthModule` tiene `imports: [UserModule, ...]`, así que `AuthService` y `JwtStrategy` pueden pedir `UserService`.

> **Excepción: módulos globales.** `ConfigModule.forRoot({isGlobal: true})` hace que `ConfigService` se pueda inyectar en **cualquier** módulo sin importarlo. Por eso `JwtStrategy` lo recibe sin que `AuthModule` tenga que importarlo para ese fin.

7. `@InjectRepository` y `forFeature`

Los repositorios de TypeORM son el caso que más confunde, porque no son clases tuyas. Hay dos piezas que van juntas:

Pieza	Qué hace	Si falta...
`TypeOrmModule.forFeature([Car])` (en el módulo)	**Crea y registra** el provider `Repository<Car>` en ese módulo, con su token.	Error: *Nest can't resolve dependencies of the CarsService (CarRepository, ...)*
`@InjectRepository(Car)` (en el constructor)	**Pide** ese provider concreto por su token.	Nest solo vería el tipo `Repository`, igual para todas las entidades: no sabría cuál darte.

¿Por qué hace falta el decorador si ya escribiste `Repository<Car>`? Porque TypeScript borra los genéricos: en ejecución `Repository<Car>` y `Repository<Brand>` son **el mismo tipo** (`Repository`). `@InjectRepository(Car)` es el que aporta la información que se perdió.

8. Otros lugares del repo donde aparece la inyección

a) Servicios que se piden entre sí — `AuthService`

```
@Injectable()
export class AuthService {
  constructor(
    private readonly userService: UserService,  // viene de UserModule (exports)
    private readonly jwtService: JwtService     // viene de JwtModule
  ){}
```

b) Una estrategia de Passport también es un provider — `JwtStrategy`

```
@Injectable()
export class JwtStrategy extends PassportStrategy(Strategy) {
  constructor(
    private readonly configService: ConfigService,  // global (isGlobal)
    private readonly userService: UserService       // UserModule (exports)
  ) {
    super({ ... secretOrKey: configService.get('JWT_SECRET') || '' });
  }
```

Está en `providers: [AuthService, JwtStrategy]`, así que Nest la instancia al arrancar (y de paso se registra en Passport bajo el nombre `'jwt'`). Luego `AuthGuard()` la encuentra *por ese nombre*, no por inyección.

c) Un guard que recibe una utilidad del propio Nest — `UserRoleGuard`

```
@Injectable()
export class UserRoleGuard implements CanActivate {
  constructor ( private readonly reflector: Reflector ){}   // viene incluido en Nest
```

En el controller solo se escribe `@UseGuards(AuthGuard(), UserRoleGuard)` con la **clase**: no se hace `new UserRoleGuard(...)`. Nest la construye y le pasa el `Reflector`.

d) Configuración que también necesita inyección — `JwtModule.registerAsync`

```
JwtModule.registerAsync({
  imports: [ConfigModule],
  inject: [ConfigService],                    // "dame estas dependencias..."
  useFactory: (configService: ConfigService) => ({   // "...y arma la configuración"
    secret: configService.get('JWT_SECRET') as string | 'secret',
    signOptions:{expiresIn: configService.get('JWT_EXPIRES_IN') | 600}
  })
})
```

`JwtModule.register({...})` recibiría valores fijos. Con `registerAsync`, la configuración depende de `ConfigService` (que lee el `.env`), así que se le indica qué inyectar (`inject`) y cómo usarlo (`useFactory`). Es la misma inyección, pero aplicada a la *configuración* de un módulo.

e) Contraejemplo: lo que NO pasa por el contenedor

```
// main.ts
app.useGlobalPipes(new ValidationPipe({ whitelist: true, forbidNonWhitelisted: true }));
```

Aquí el `ValidationPipe` se crea con `new` a mano: es un objeto que **tú** construyes y le entregas a Nest. Funciona porque no necesita dependencias; si las necesitara, tendrías que registrarlo de otra forma para que el contenedor se las inyecte.

9. Mapa completo: quién recibe qué en el repo

Clase (consumidor)	Recibe	Origen
`AppController`	`AppService`	`providers` de AppModule
`BrandsController`	`BrandsService`	`providers` de BrandsModule
`BrandsService`	`Repository<Brand>`	`forFeature([Brand])`
`CarsController`	`CarsService`	`providers` de CarsModule
`CarsService`	`Repository<Car>` `BrandsService`	`forFeature([Car])` BrandsModule → `exports`
`UserController`	`UserService`	`providers` de UserModule
`UserService`	`Repository<User>`	`forFeature([User])`
`AuthController`	`AuthService`	`providers` de AuthModule
`AuthService`	`UserService` `JwtService`	UserModule → `exports` `JwtModule.registerAsync`
`JwtStrategy`	`ConfigService` `UserService`	ConfigModule (global) UserModule → `exports`
`UserRoleGuard`	`Reflector`	Incluido en `@nestjs/core`

10. Errores de inyección: qué significan

Todos son errores de **arranque** (la app no llega a abrir el puerto). El mensaje suele decir el servicio afectado, el **índice** del parámetro que falló (`[0]`, `[1]`...) y muestra un ? en la posición de la dependencia que no encontró.

Qué olvidaste	Síntoma	Arreglo
`exports: [BrandsService]` en BrandsModule	*Nest can't resolve dependencies of the CarsService (CarRepository, ?)*	Añadir el `exports`
`imports: [BrandsModule]` en CarsModule	El mismo mensaje	Añadir el `imports`
`TypeOrmModule.forFeature([Car])`	*... (?, BrandsService)*: no encuentra `CarRepository`	Añadir `forFeature` en el módulo del service
Poner el service en `providers`	El controller no encuentra su service	Registrarlo en `providers`
`@Injectable()` en una clase con dependencias	Nest no ve los tipos del constructor	Añadir el decorador
Dependencia circular (A importa B y B importa A)	Error de dependencia circular	Rediseñar a un solo sentido, o `forwardRef`

11. Para qué sirve todo esto: pruebas y cambios

La gran ventaja práctica es que `CarsService` no sabe de dónde viene su repositorio. En un test puedes darle uno falso. (Ejemplo ilustrativo mío, **no** es código del repo: los `.spec.ts` que hay los genera el CLI.)

```
ejemplo ilustrativo de prueba unitaria

const module = await Test.createTestingModule({
  providers: [
    CarsService,
    { provide: getRepositoryToken(Car), useValue: { find: jest.fn() } },  // falso
    { provide: BrandsService,           useValue: { findOne: jest.fn() } },  // falso
  ],
}).compile();

const service = module.get(CarsService);  // CarsService real, dependencias falsas
```

Lo mismo pasa con los cambios: si mañana `BrandsService` necesita otra dependencia, solo se toca su constructor y su módulo; nadie más se entera.

12. Resumen para llevarte

- **Pedir, no crear:** en el constructor declaras lo que necesitas; Nest lo entrega.
- **Tres piezas:** provider (`@Injectable` + `providers`), consumidor (constructor) y contenedor (Nest).
- **Cómo lo sabe:** `emitDecoratorMetadata` conserva los tipos del constructor; Nest los lee.
- **Módulos = visibilidad:** para usar el service de otro módulo, ese lo `exports` y tú lo `imports`.
- **Repositorios:** `forFeature([Entidad])` los crea, `@InjectRepository(Entidad)` los pide.
- **Singleton:** una sola instancia compartida, creada al arrancar.
- **Errores de inyección** = arranque, mensaje con ?; casi siempre falta un `exports`, un `imports`, un `forFeature` o un `providers`.

> Para practicarlo: el **Cuestionario 2** (Módulos e inyección de dependencias) de tu carpeta `compunet/NestJS` repasa todo esto con huecos, retos y casos de error.